

Decomposição de Benders

Conversa exportada · comunicação mestre-subproblema e o caso inteiro

PERGUNTA

Como funciona a comunicação entre o problema mestre e o subproblema, na decomposição de Benders?

RESPOSTA

A decomposição de Benders funciona como um diálogo iterativo entre os dois problemas, e a chave para entender essa comunicação é saber exatamente *o que* cada lado envia ao outro.

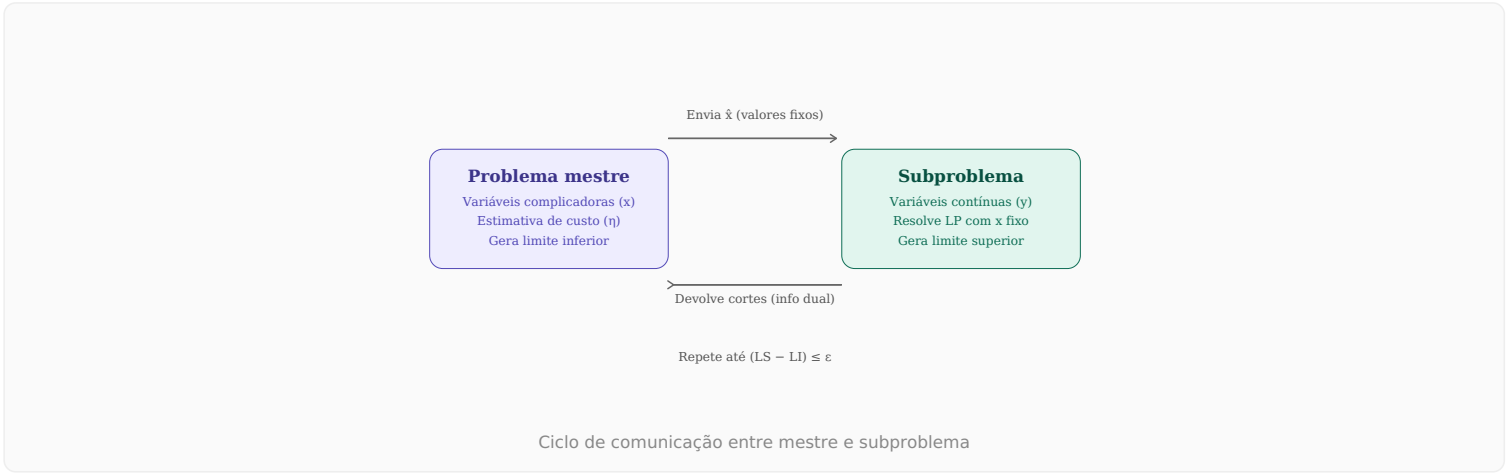
O ponto de partida é a separação das variáveis. O **problema mestre** fica com as variáveis “complicadoras” (tipicamente as inteiras, ou aquelas que, uma vez fixadas, tornam o resto fácil — chame-as de x). O **subproblema** fica com as variáveis restantes (contínuas, y), resolvidas *supondo x fixo*. Considerando um problema de minimização:

$$\min c^T x + d^T y, \text{ sujeito a } Ax = b, Bx + Dy = f, x \in X, y \geq 0$$

A comunicação acontece em duas direções, a cada iteração:

Mestre → subproblema: o mestre resolve sua versão relaxada e produz um candidato $\hat{x}$ para as variáveis complicadoras, além de uma variável auxiliar η que *estima* o custo do subproblema. Ele envia esse $\hat{x}$ para o subproblema. Nesse momento, $c^T \hat{x} + \eta$ é um **limite inferior** para o valor ótimo.

Subproblema → mestre: com x fixo, o subproblema vira um LP. Ele não devolve “a resposta” diretamente — devolve **cortes**, construídos a partir da informação *dual*.



O ponto central é **por que os cortes funcionam**. Quando você fixa $x = \hat{x}$, o subproblema primal é um LP. Considere seu dual:

$$\max (f - B\hat{x})^T u, \text{ sujeito a } D^T u \leq d$$

A região viável do dual, $\{u : D^T u \leq d\}$, **não depende de $\hat{x}$** — só a função objetivo depende. Logo, os pontos extremos e raios extremos desse poliedro dual são fixos ao longo de todas as iterações. É essa propriedade que permite ao subproblema traduzir sua resposta numa restrição válida sobre as variáveis do mestre.

Subproblema viável e limitado. O dual devolve um ponto extremo u_p . Se a estimativa η do mestre for menor que $(f - B\hat{x})^T u_p$, ela estava otimista demais, e envia-se um **corte de otimalidade**: $\eta \geq (f - B\hat{x})^T u_p$.

Subproblema inviável. O dual é ilimitado e devolve um raio extremo r_q . Envia-se um **corte de viabilidade**: $0 \geq (f - B\hat{x})^T r_q$, que elimina do mestre a região de x que causaria a mesma inviabilidade.

A convergência acontece porque o **limite inferior** vem do mestre relaxado ($c^T \hat{x} + \eta$) e cresce monotonicamente, enquanto o **limite superior** vem de qualquer solução viável do subproblema. Quando os dois se aproximam a menos de uma tolerância ϵ , tem-se o ótimo.

Em resumo: o mestre manda *uma proposta de decisão*, e o subproblema responde com *uma desigualdade linear derivada da dualidade* — seja para corrigir o custo estimado, seja para proibir uma decisão inviável.

PERGUNTA

E quando o problema é inteiro (binário), o que muda?

RESPOSTA

A resposta depende inteiramente de **onde** estão as variáveis inteiras — e essa distinção é a coisa mais importante a fixar, porque os dois casos são radicalmente diferentes.

Caso 1 — inteiros no mestre (o mais comum)

Se as variáveis complicadoras x são binárias/inteiras mas o subproblema continua sendo um LP contínuo, então **a teoria dos cortes não muda em nada**. Ao fixar $x = x$, o subproblema ainda é um LP com x entrando apenas como constante no lado direito. A dualidade continua valendo, os pontos e raios extremos do poliedro dual continuam fixos, e os cortes de otimalidade e viabilidade são construídos exatamente como antes.

O que muda é o **mestre**: ele deixa de ser um LP e passa a ser um MILP (NP-difícil). Isso dá origem a duas formas de implementar:

A forma “de livro-texto” resolve o MILP do mestre *até a otimalidade*, adiciona o corte, e resolve o MILP inteiro de novo do zero — a cada iteração. Isso é caríssimo, porque você paga um branch-and-bound completo por iteração.

A forma moderna, chamada **branch-and-Benders-cut** (branch-and-cut com cortes de Benders), resolve **uma única árvore** de branch-and-bound. Sempre que o solver encontra um candidato x inteiro em algum nó, resolve-se o subproblema e, se o corte for violado, adiciona-se como uma *lazy constraint*. É como os solvers comerciais (CPLEX, Gurobi via callbacks) fazem, e é ordens de magnitude mais rápido.

Uma vantagem colateral do caso binário: como x é discreto e finito, além dos cortes duais há sempre, como último recurso, os **cortes combinatórios** (ou *no-good cuts*), que no mínimo eliminam exatamente o x já testado.

Caso 2 — inteiros no subproblema (aqui tudo quebra)

Se as variáveis do subproblema (o segundo estágio, o *recurso* y) são inteiras, o subproblema vira um MILP — e **a base inteira do método deixa de funcionar**:

Não há mais dualidade forte de LP. A função de valor $Q(x)$ do subproblema deixa de ser convexa: passa a ser descontínua e cheia de “degraus”. Os cortes clássicos de Benders são hiperplanos de suporte de uma função convexa — aplicados a uma função não-convexa, eles **cortam soluções ótimas legítimas** e produzem respostas erradas.

Nesse cenário é preciso trocar o tipo de corte:

- O **método L-shaped inteiro** (Laporte & Louveaux, 1993) exige primeiro estágio binário. Resolve o subproblema inteiro até a otimalidade e constrói cortes exatos no ponto x visitado, recuando para um limite inferior L nos demais pontos — combinando um corte tipo *no-good* com um piso válido sobre o recurso. Os cortes são localizados, o que pode exigir muitas iterações.
- A **decomposição de Benders lógica** (Hooker & Ottosson) generaliza a ideia usando um *dual de inferência*: o subproblema pode ser qualquer problema {programação por restrições, escalonamento, roteamento}, e o corte vem de um argumento lógico sobre *por que* aquele x foi ruim, não de multiplicadores duais. Muito usada em scheduling e alocação.
- Os **cortes combinatórios de Benders** (Codato & Fischetti), voltados a MILPs onde o acoplamento é por implicações lógicas, com cortes baseados em subsistemas inviáveis mínimos.

Resumindo

Inteiros **no mestre** mudam o *custo computacional* e a *estratégia de implementação* (vá de branch-and-Benders-cut), mas não a matemática dos cortes. Inteiros **no subproblema** mudam a *própria validade dos cortes* e exigem métodos especializados (L-shaped inteiro, Benders lógico), porque a dualidade de LP — o alicerce de tudo — desaparece.